

NOME: Beatriz Carvalho Sousa

RA: 082230027

TURMA: EC6

NOME: Fernando Montanher da Cruz

RA: 082230010

NOME: Lucas da Silva Macêdo

RA: 082230037

NOME: Rhyú Costa de Souza

RA: 082230023

NOME: Yuri Villaço de Souza

RA: 082230036

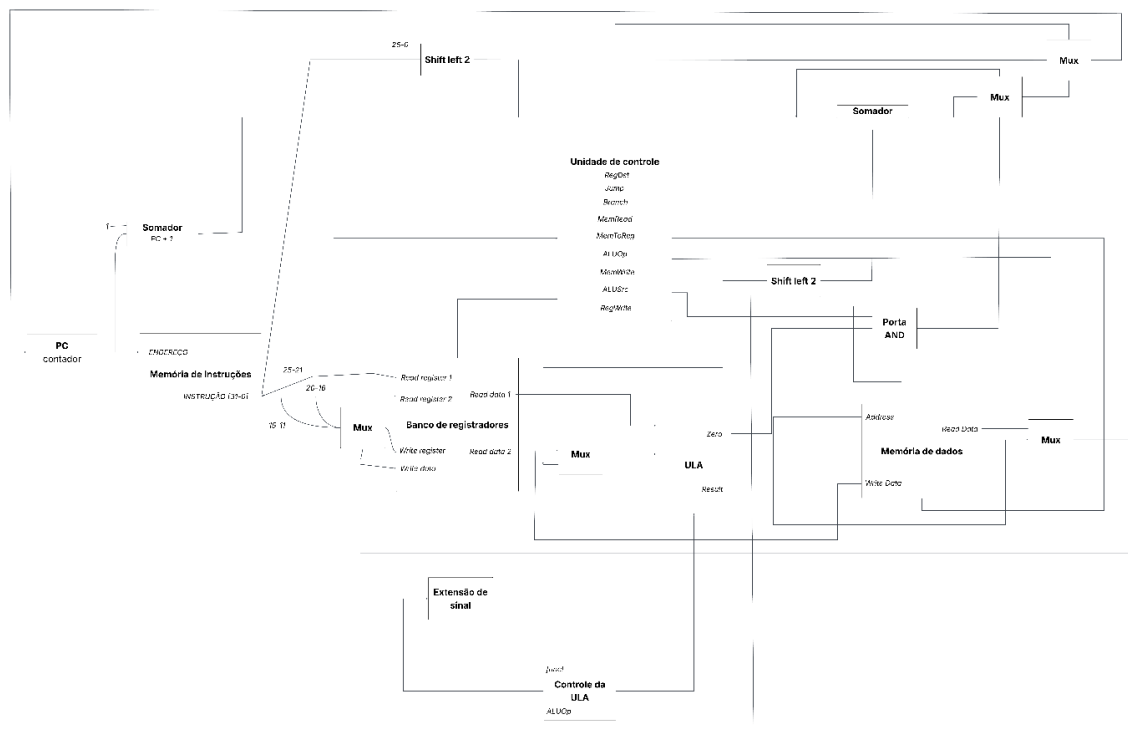
DOCUMENTAÇÃO TÉCNICA

Introdução

O processador apresentado implementa uma versão simplificada da arquitetura MIPS de 32 bits, projetada para fins de aprendizado. Cada instrução é executada em um único ciclo de clock, sem pipeline. O circuito é composto por diversos módulos: PC, Memória de Instruções, Unidade de Controle, Banco de Registradores, ULA, Memória de Dados e lógica de desvio/salto, todos desenvolvidos no software Logisim.

Diagrama de blocos

O presente diagrama busca representar a lógica implementada no software, trazendo as conexões entre os módulos e outros componentes, como portas lógicas e multiplexadores. Ele pode ser observado também por meio do repositório do GitHub.



Descrição dos módulos

O circuito conta com os seguintes módulos para seu desenvolvimento.

Contador de Programa (PC)

Um registrador de 32 bits que armazena o endereço da instrução corrente. A cada ciclo de clock, é atualizado com o valor selecionado pelo MUX de salto — podendo receber PC+4 (fluxo sequencial), o endereço de branch (desvio condicional) ou o endereço de jump (salto incondicional). Possui sinal de clear para inicialização.

Memória de instruções

Memória somente de leitura que fornece a instrução de 32 bits apontada pelo PC. A saída é dividida entre os campos: opcode [31–26], rs [25–21], rt [20–16], rd [15–11], shamt [10–6], funct [5–0] e imediato [15–0].

Somador PC+4

É um circuito combinacional que incrementa o PC em 4 a cada ciclo, mantendo o fluxo sequencial das instruções.

Unidade de controle

Faz a decodificação do campo opcode (6 bits) e gera todos os sinais de controle do caminho de dados: RegDst, RegWrite, ALUSrc, MemWrite, MemRead, MemToReg, Branch, Jump e ALUOp, tendo sido implementada com lógica combinacional (portas AND/OR apresentadas no circuito).

Banco de registradores

Contém 8 registradores de 32 bits (R0–R7), tendo duas portas de leitura simultâneas (Read data 1 e 2) e uma porta de escrita controlada por RegWrite e Clock. Um DEMUX roteia o dado de escrita e o sinal de habilitação ao registrador correto, enquanto dois MUX selecionam os registradores de leitura.

Controle da ULA

Recebe ALUOp (da unidade de controle) e o campo funct (da instrução) e gera o sinal ULACtrl de 3 bits que seleciona a operação a ser executada pela ULA via MUX interno.

ULA (Unidade Lógico-Aritmética)

Executa as operações: soma, subtração, multiplicação, divisão, negação, comparação, shift à esquerda e shift à direita. Internamente, dois DEMUX roteiam as entradas A e B para as unidades funcionais e um MUX seleciona o resultado. Produz também o sinal Zero/Igual, utilizado pelo branch.

Extensão de sinal

Converte o imediato de 16 bits (instrução[15–0]) para 32 bits com extensão de sinal, que é necessário para instruções do tipo I (lw, sw, beq, addi).

Memória de dados

Memória de leitura/escrita acessada pela ULA (endereço) e pelo banco de registradores (dado de escrita), controlada por MemRead e MemWrite. O dado lido retorna ao banco via MUX MemToReg.

Lógica de desvio e salto

O endereço de branch é calculado somando PC+4 ao imediato deslocado 2 bits à esquerda. Uma porta AND combina o sinal Branch com o sinal Zero da ULA. O endereço de jump concatena os bits [27–0] da instrução (deslocados) com os 4 bits altos de PC+4 e um MUX de dois estágios seleciona o próximo PC.

Decisões de projeto e suas justificativas técnicas

Para melhor desenvolvimento e estruturação do projeto, houve a aplicação dos conceitos apresentados abaixo, junto de suas respectivas justificativas.

Arquitetura monociclo - Cada instrução executa em um único ciclo de clock. Dessa forma, há simplificação do controle (sem pipeline), auxiliando para fins didáticos. O ponto negativo é que o ciclo precisa acomodar a instrução mais lenta (lw), o que limita a frequência de operação.

8 registradores de propósito geral - Redução da complexidade do banco de registradores sem comprometer a implementação. Desse modo, no circuito exige apenas 3 bits nos campos rs, rt e rd da instrução.

Unidade de controle combinacional - Implementada com portas lógicas AND/OR diretamente sobre o opcode. Essa abordagem é eficiente para poucos tipos de instrução, evitando o custo de uma memória de controle microprogramada.

Separação entre Controle da ULA e Unidade de Controle - Segue o modelo clássico do MIPS, onde a unidade de controle principal determina o tipo de operação (via ALUOp) e o controle da ULA refina a operação exata com base no campo funct. Dessa forma, permite que instruções R-type utilizem diversas operações sem ampliar o opcode.

MUX de dois estágios para o próximo PC - Primeiro seleciona entre PC+4 e o endereço de branch (via AND com Zero); em seguida, seleciona entre esse resultado e o endereço de jump. Essa hierarquia garante que jump tem prioridade sobre Branch, por ser uma instrução incondicional.

Extensão de sinal em 16→32 bits - Necessária para que imediatos negativos funcionem corretamente em instruções aritméticas e de desvio. A extensão por sinal preserva o valor em complemento de dois.

Fluxo de dados

O fluxo de dados a seguir corresponde ao caminho que a informação percorre dentro do processador durante a execução de uma instrução.

Etapa 1 - Busca da Instrução: O PC (Contador de Programa) fornece o endereço da instrução corrente, enviado à Memória de Instruções para retornar uma palavra de 32 bits. Assim, a instrução é então fatiada em seus campos (opcode, registradores, imediato, funct), que alimentam diferentes módulos.

Etapa 2 - Decodificação e Controle: O opcode é interpretado pela Unidade de Controle, que gera sinais de controle para todo o datapath. Esses, por sua vez, determinam se haverá escrita em registradores, acesso à memória, operação da ULA ou atualização do PC por branch/jump. Em paralelo, o campo funct é enviado ao Controle da ULA, que refina a operação aritmética/lógica.

Etapa 3 - Leitura dos Operandos: Os campos rs e rt selecionam dois registradores no Banco de Registradores. Os valores lidos (Read Data 1 e Read Data 2) são encaminhados à ULA ou à Memória de Dados. Se a instrução for do tipo I, o campo imediato é expandido de 16 para 32 bits pela Extensão de Sinal.

Etapa 4 - Execução na ULA: A ULA recebe os operandos e o sinal de controle (ULACtrl) e executa a operação solicitada: soma, subtração, comparação, deslocamento ou qualquer outra. Após isso, ela produz dois resultados, o result (valor de saída da operação) e o zero (indicador de igualdade, usado em branches).

Etapa 5 - Acesso à Memória de Dados: Para instruções de carga/armazenamento (lw, sw), o resultado da ULA é usado como endereço. O dado pode ser escrito na memória (MemWrite) ou lido dela (MemRead). O valor lido retorna ao banco de registradores via MUX MemToReg.

Etapa 6 - Atualização do PC: O próximo valor do PC é escolhido por uma hierarquia de MUX, onde o PC+4 tem execução sequencial; branch é calculado como PC+4 + imediato deslocado 2 bits, sendo ativado se branch=1 e zero=1; e, jump, em que o endereço é formado pela concatenação dos bits da instrução com os 4 bits mais altos de PC+4.